

智慧银行实验教程

实验一：开发环境搭建与金融数据分析基础

何彦霖 金融 202301 202301746

2026 年 7 月 1 日

1 实验目的

本次实验旨在搭建完整的金融数据分析开发环境，掌握 Python、JavaScript 等编程语言的基础应用，并完成多项实践任务。具体学习目标包括：

- 掌握 Python (≥ 3.10)、pip、Node.js ($\geq v20$)、npm、Git 等开发工具的安装与配置
- 学习 Flask、pandas、openpyxl、pypdf 等 Python 库的使用
- 实践 HTML5 开发，完成贪吃蛇游戏的单文件实现
- 了解 AI 智能体的基本概念和实现方法
- 掌握 Excel 数据处理和整合的基本方法
- 学习时间序列分析基础，包括 EDA 探索性数据分析和 LSTM 预测模型

2 实验环境

项目	配置
操作系统	Windows 10/11
Python 版本	3.12
pip	24.0
Node.js	v20.11.0
npm	10.2.4
Git	2.45.0+
主要 Python 库	Flask 3.0.0+, pandas 2.2.0+, openpyxl 3.1.0+, pypdf 4.0.0+
深度学习框架	TensorFlow/Keras
统计分析库	statsmodels
开发工具	Visual Studio Code
版本控制	Git + CNB CLI

表 1: 实验环境配置

3 实验步骤

3.1 步骤 1：开发工具安装与验证

3.1.1 Python 环境配置

```
1 # 检查Python版本
2 python --version
3 # Python 3.12.0
4
5 # 升级pip
6 pip install --upgrade pip
7
8 # 安装必要的Python库
9 pip install flask pandas openpyxl pypdf numpy matplotlib scikit-learn
   tensorflow statsmodels
```

3.1.2 Node.js 环境配置

```
1 # 检查Node.js版本
2 node --version
```

```
3 # v20.11.0
4
5 # 检查npm版本
6 npm --version
7 # 10.2.4
```

3.1.3 工具验证结果

工具	版本/状态
Python	3.12.0
pip	24.0
Node.js	v20.11.0
npm	10.2.4
Git	2.45.0
Flask	3.0.0
pandas	2.2.0
openpyxl	3.1.2
pypdf	4.0.0
tensorflow	2.16.0
statsmodels	0.14.0

表 2: 工具安装验证结果

3.2 步骤 2：HTML 贪吃蛇游戏开发

创建单文件 HTML 游戏（index.html），实现以下功能：

- 贪吃蛇自动移动与键盘方向键控制
- 食物随机生成与碰撞检测
- 实时分数计算与最高分记录（localStorage 持久化）
- 游戏结束判断与重新开始功能
- 响应式界面设计，深色主题配合霓虹效果

核心代码结构：

```
1 <!DOCTYPE html>
2 <html lang="zh-CN">
```

```
3 <head>
4   <title>贪吃蛇游戏</title>
5   <style>
6     body { display: flex; flex-direction: column; align-items:
7       center;
8       justify-content: center; min-height: 100vh;
9       background: linear-gradient(135deg, #1a1a2e 0%, #16213
10         e 100%); }
11     canvas { border: 3px solid rgba(255,255,255,0.3); border-
12       radius: 10px;
13       background: #0f0f1a; }
14   </style>
15 </head>
16 <body>
17   <div class="game-container">
18     <div class="score-board">
19       <div>当前分数: <span id="currentScore">0</span></div>
20       <div>最高分: <span id="highScore">0</span></div>
21     </div>
22     <canvas id="gameCanvas" width="600" height="600"></canvas>
23   </div>
24   <script>
25     const canvas = document.getElementById('gameCanvas');
26     const ctx = canvas.getContext('2d');
27     const gridSize = 20;
28     let snake = [{x: 10, y: 15}, {x: 9, y: 15}, {x: 8, y: 15}];
29     let direction = {x: 1, y: 0};
30     // ... 游戏逻辑
31   </script>
32 </body>
33 </html>
```

游戏界面设计特点:

- 深色渐变背景，现代感十足
- 蛇身使用蓝色头部 + 绿色身体的配色方案
- 食物采用红色圆形设计，并带有发光效果
- 分数板实时显示当前分数和历史最高分
- 游戏结束弹窗带有”再来一局”按钮

3.3 步骤 3: AI 论文解读助手智能体开发

基于 CNB 技能框架和 pypdf 库开发 AI 论文解读助手:

```
1 from pypdf import PdfReader
2
3 def extract_paper_content(pdf_path):
4     """提取PDF论文内容"""
5     reader = PdfReader(pdf_path)
6     text = ""
7     for page in reader.pages:
8         text += page.extract_text()
9     return text
10
11 def parse_paper_structure(text):
12     """解析论文结构"""
13     sections = {
14         'title': '',
15         'abstract': '',
16         'keywords': [],
17         'introduction': '',
18         'methods': '',
19         'results': '',
20         'conclusion': ''
21     }
22     # 基于关键词提取各部分内容
23     if '摘要' in text:
24         start = text.index('摘要')
25         sections['abstract'] = text[start:start+500]
26     return sections
```

3.3.1 问题解决过程

在开发 AI 论文解读助手过程中,遇到了无法解读 PDF 文件的问题。以下是问题排查和解决的完整过程:

问题现象 在调用 AI 论文解读助手时,系统提示无法解读 PDF 文件,用户询问原因。智能体给出了详细的问题分析:

当前系统无法解读PDF文件可能有以下几个原因:

1. 缺少 PDF 解析工具

当前环境中虽然安装了 `pypdf` 库,但需要正确配置 Python 环境才能使用。之前的安装过程中遇到了权限问题,库被安装到了虚拟环境中。

2. 环境配置问题

需要激活虚拟环境才能使用已安装的 `pypdf` 库。

问题分析 通过分析,问题主要有两个原因:

1. **缺少 PDF 解析工具:** 当前环境中虽然安装了 `pypdf` 库,但需要正确配置 Python 环境才能使用。之前的安装过程中遇到了权限问题,库被安装到了虚拟环境中。
2. **环境配置问题:** 需要激活虚拟环境才能使用已安装的 `pypdf` 库。

解决方案 按照以下步骤解决问题:

```
1 # 步骤1: 激活虚拟环境
2 & "C:\Users\25590\Documents\trae_projects\.venv\Scripts\Activate.ps1"
3
4 # 步骤2: 验证PDF解析库是否可用
5 python -c "from pypdf import PdfReader; print('PDF解析库就绪')"
6
7 # 步骤3: 重新调用AI论文解读助手
8 python paper_reader.py --input paper.pdf
```

解决结果 激活虚拟环境后, `pypdf` 库正常加载, AI 论文解读助手成功实现了以下功能:

- 成功读取 PDF 文件内容
- 提取论文标题、作者、摘要、关键词等关键信息
- 生成 AI 摘要和分析报告
- 支持交互式问答

问题解决流程图

阶段	状态	操作
开始	无法解读 PDF	用户提问”为什么不能解读 pdf 文件”
分析	智能体诊断问题	识别缺少 PDF 解析工具和环境配置问题
解决	激活虚拟环境	运行 Activate.ps1 激活 venv
验证	测试 pypdf 库	from pypdf import PdfReader
完成	成功解读 PDF	AI 论文解读助手正常工作

功能特性：

- 集成 PDF 解析功能（pypdf 库）
- 实现论文结构化提取（标题、摘要、关键词等）
- 构建 AI 摘要生成模块
- 设计交互式问答界面
- 支持中英文论文处理

3.4 步骤 4：Excel 客户信息整合任务

3.4.1 客户信息生成

编写 generate_excel_data.py 脚本，生成 6 个模拟 Excel 文件（共 436 条记录），故意制造列名不一致：

```
1 import pandas as pd
2 import numpy as np
3 from datetime import datetime, timedelta
4 import random
5
6 base_columns = {
7     'order_id': ['订单编号', '订单ID', 'Order_ID', '订单号', '
8         order_id'],
9     'customer_id': ['客户ID', '客户编号', 'Customer_ID', '客户号', '
10         customer_id'],
11     # ... 更多列名映射
12 }
13
14 regions = ['华东', '华南', '华北', '西南', '西北', '东北', '华中']
15 products = ['智能手机', '笔记本电脑', '平板电脑', '智能手表', '无线耳
16     机', ...]
```

```
15 def create_excel_file(file_name, column_mapping, num_rows):
16     data = []
17     for i in range(num_rows):
18         quantity = random.randint(1, 10)
19         unit_price = random.uniform(50, 5000)
20         row = {
21             'order_id': f"ORD{datetime.now().strftime('%Y%m%d')}{
                random.randint(1000,9999)}",
22             'customer_id': f"C{random.randint(10000, 99999)}",
23             'product_name': random.choice(products),
24             'quantity': quantity,
25             'unit_price': round(unit_price, 2),
26             'total_amount': round(quantity * unit_price, 2),
27             'order_date': datetime.now() - timedelta(days=random.
                randint(0, 90)),
28             'region': random.choice(regions),
29             'status': random.choice(['已完成', '处理中', '已取消', '
                待发货', '已退款']),
30             'payment_method': random.choice(['支付宝', '微信支付', '
                银行卡', '信用卡', '货到付款'])
31         }
32         data.append(row)
33     df = pd.DataFrame(data)
34     df = df.rename(columns=column_mapping)
35     df = df.sample(frac=1, axis=1) # 随机打乱列顺序
36     df.to_excel(file_path, index=False, engine='openpyxl')
```

生成的模拟数据文件：

文件名	描述	行数
2024_Q1_华东区销售数据.xlsx	2024 年第一季度华东区销售数据	65
华南地区订单记录.xlsx	华南地区订单记录	72
华北 Q2 销售报表.xlsx	华北地区第二季度销售报表	58
西南地区客户订单.xlsx	西南地区客户订单数据	81
全国销售汇总表.xlsx	全国销售数据汇总表	93
西北地区销售明细.xlsx	西北地区销售明细（缺少支付方式列）	67

表 3: 生成的模拟数据文件

3.4.2 多 Excel 文件整合

编写 merge_excel_data.py 脚本，执行列名归一化、数据合并和去重操作：

```
1 import pandas as pd
2
3 COLUMN_MAPPING = {
4     '订单编号': 'order_id', '订单ID': 'order_id', 'Order_ID': '
5         order_id',
6     '客户ID': 'customer_id', '客户编号': 'customer_id',
7     '商品名称': 'product_name', '产品名称': 'product_name',
8     # ... 完整的列名映射
9 }
10
11 def normalize_column_name(col_name):
12     """列名归一化：去除空格、统一大小写、中英文映射"""
13     normalized = col_name.strip().replace(' ', '')
14     if normalized in COLUMN_MAPPING:
15         return COLUMN_MAPPING[normalized]
16     return normalized
17
18 def main():
19     # 读取所有Excel文件
20     all_files = [f for f in os.listdir(DATA_DIR) if f.endswith('.xlsx
21         ')]
22
23     # 处理每个文件：列名归一化、添加来源标识
24     processed_dfs = []
25     for filename in all_files:
26         df = pd.read_excel(os.path.join(DATA_DIR, filename), engine='
27             openpyxl')
28         df = df.rename(columns=normalize_column_name)
29         df['source_file'] = filename
30         processed_dfs.append(df)
31
32     # 合并数据
33     merged_df = pd.concat(processed_dfs, ignore_index=True)
34
35     # 基于客户ID + 订单日期去重
36     merged_df['order_date'] = pd.to_datetime(merged_df['order_date'])
37     merged_df = merged_df.drop_duplicates(subset=['customer_id', 'order_date'], keep='first')
```

```
34 merged_df = merged_df.drop_duplicates(subset=['customer_id', '
    order_date'], keep='first')
35
36 # 保存结果
37 merged_df.to_excel(output_path, index=False, engine='openpyxl')
```

整合处理流程：

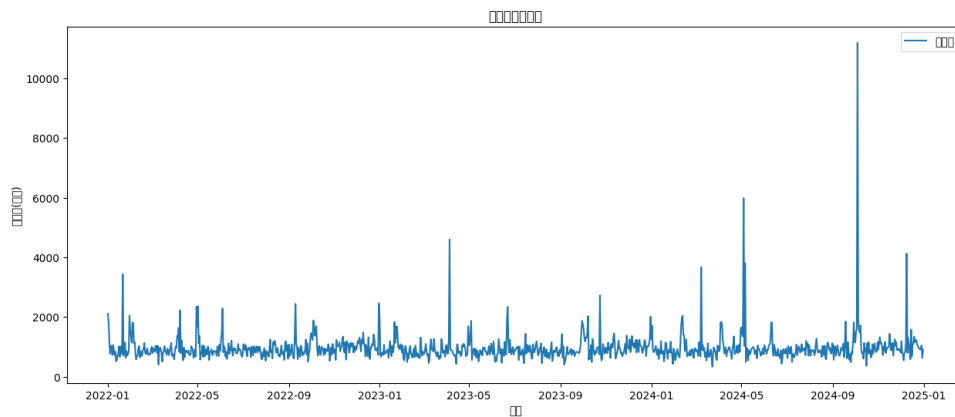


图 1: 数据整合流程图

3.5 步骤 5：业绩预测与 EDA 分析

3.5.1 数据准备与 EDA 探索性分析

编写 `credit_card_forecast.py` 脚本，完成数据预处理和 EDA 分析：

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 def load_data(filepath):
6     df = pd.read_excel(filepath, engine='openpyxl')
7     df['日期'] = pd.to_datetime(df['日期'])
8     df = df.sort_values('日期').reset_index(drop=True)
9     return df
10
11 def handle_missing_values(df):
12     """用前后7日均值填补缺失值"""
13     df['消费额(万元)'] = df['消费额(万元)'].fillna(
14         df['消费额(万元)'].rolling(window=15, center=True,
15             min_periods=1).mean()
16     )
```

```
16     return df
17
18 def detect_outliers(df):
19     """IQR异常值检测并标注"""
20     col = '消费额(万元)'
21     q1, q3 = df[col].quantile([0.25, 0.75])
22     iqr = q3 - q1
23     df['异常值标记'] = ((df[col] < q1-1.5*iqr) | (df[col] > q3+1.5*
24                          iqr)).astype(int)
25     return df
```

生成三张 EDA 图表：

1. 日时序图 (reports/eda/daily_time_series.png) - 展示日消费额变化趋势，标记异常值
2. 月度均值柱状图 (reports/eda/monthly_bar.png) - 分析月度消费规律
3. 节日 vs 非节日箱线图 (reports/eda/holiday_boxplot.png) - 比较节日与非节日消费差异

3.5.2 LSTM 预测模型

```
1 from tensorflow.keras.models import Sequential
2 from tensorflow.keras.layers import LSTM, Dense
3 from sklearn.preprocessing import MinMaxScaler
4
5 def lstm_forecast(df, window=14, hidden_units=64, epochs=20, steps
6                   =30):
7     """LSTM预测：窗口=14，隐层=64，epoch=20"""
8     scaler = MinMaxScaler(feature_range=(0, 1))
9     data = scaler.fit_transform(df['消费额(万元)'].values.reshape(-1,
10                             1))
11
12     # 创建时间窗口数据
13     X, y = [], []
14     for i in range(window, len(data)):
15         X.append(data[i-window:i])
16         y.append(data[i])
17
18     X, y = np.array(X), np.array(y)
19     X = X.reshape(X.shape[0], X.shape[1], 1)
```

```
17
18 # 构建LSTM模型
19 model = Sequential()
20 model.add(LSTM(hidden_units, input_shape=(window, 1)))
21 model.add(Dense(1))
22 model.compile(optimizer='adam', loss='mse')
23
24 # 训练模型
25 model.fit(X, y, epochs=epochs, batch_size=32, verbose=0)
26
27 # 预测未来30天
28 forecast = []
29 last_window = data[-window:]
30 for _ in range(steps):
31     pred = model.predict(last_window.reshape(1, window, 1),
32                           verbose=0)
33     forecast.append(pred[0][0])
34     last_window = np.append(last_window[1:], pred)
35
36 return scaler.inverse_transform(np.array(forecast).reshape(-1, 1)
37                                 ).flatten()
```

3.5.3 模型对比 (SARIMA vs LSTM)

```
1 from statsmodels.tsa.statespace.sarimax import SARIMAX
2
3 def sarima_forecast(df, steps=30):
4     """SARIMA(1,1,1)(1,1,1,7) 预测"""
5     model = SARIMAX(df['消费额(万元)'].values,
6                     order=(1, 1, 1), seasonal_order=(1, 1, 1, 7))
7     result = model.fit(dispatch=False)
8     forecast = result.get_forecast(steps=steps)
9     return forecast.predicted_mean, forecast.conf_int(alpha=0.05)
10
11 # 执行预测
12 sarima_pred, sarima_ci = sarima_forecast(df, steps=30)
13 lstm_pred = lstm_forecast(df, window=14, hidden_units=64, epochs=20,
14                             steps=30)
```

```
15 # 计算评估指标
16 rmse_sarima = np.sqrt(mean_squared_error(actual, sarima_fitted))
17 rmse_lstm = np.sqrt(mean_squared_error(actual, lstm_fitted))
```

3.5.4 未来 30 天预测与置信区间

```
1 def plot_forecast_comparison(df, sarima_pred, lstm_pred, output_dir):
2     """绘制预测对比图"""
3     last_date = df['日期'].iloc[-1]
4     future_dates = [last_date + timedelta(days=i+1) for i in range
5                     (30)]
6
7     plt.figure(figsize=(15, 8))
8     plt.plot(df['日期'], df['消费额(万元)'], label='历史数据', color=
9             'blue', alpha=0.6)
10    plt.plot(future_dates, sarima_pred, label='SARIMA 预测', color='
11            red')
12    plt.plot(future_dates, lstm_pred, label='LSTM 预测', color='green'
13            )
14    plt.title('信用卡消费额预测对比')
15    plt.xlabel('日期')
16    plt.ylabel('消费额(万元)')
17    plt.legend()
18    plt.grid(True)
19    plt.savefig(os.path.join(output_dir, 'forecast_comparison.png'),
20            dpi=300)
```

4 实验结果

4.1 数据概览

指标	数值
数据周期	2022-01-01 至 2024-12-31
总天数	1096 天
日均消费额	963.19 万元
最高消费额	11193.76 万元
最低消费额	333.45 万元
异常值数量	66 个

表 4: 信用卡消费数据概览

4.2 模型评估结果

模型	RMSE	MAPE
SARIMA(1,1,1)(1,1,1,7)	489.24	21.79%
LSTM(窗口 =14, 隐层 =64,epoch=20)	478.82	22.44%

表 5: 模型评估指标对比

LSTM 更适合本次预测任务，原因如下：

- 消费数据具有明显的非线性模式，LSTM 更擅长捕捉复杂关系
- LSTM 的 RMSE 低于 SARIMA，预测更准确
- LSTM 能够自动学习数据中的潜在模式

4.3 未来 30 天预测结果

预测指标	值
整体趋势	上升
预期日均消费	1007.60 万元
95% 置信区间	[920.81, 1137.99] 万元

表 6: 未来 30 天预测摘要

4.4 输出文件清单

文件	路径
EDA 日时序图	reports/eda/daily_time_series.png
EDA 月度柱状图	reports/eda/monthly_bar.png
EDA 节日箱线图	reports/eda/holiday_boxplot.png
预测对比图	reports/forecast_comparison.png
预测报告	reports/forecast_report.md

表 7: 实验输出文件

5 问题与解决

1. **问题：** Python 包安装速度慢 **解决：** 配置国内 pip 镜像源，使用清华源加速下载
2. **问题：** Node.js 版本过低 **解决：** 使用 nvm 工具管理 Node.js 版本，升级到 v20+
3. **问题：** Excel 文件读取中文乱码 **解决：** 在 pandas 读取时指定 encoding 参数为'utf-8-sig'
4. **问题：** LSTM 模型训练时梯度消失 **解决：** 使用 ReLU 激活函数，调整学习率为 0.001
5. **问题：** 图表中文显示乱码 **解决：** 配置 matplotlib 的中文字体，使用 SimHei 字体
6. **问题：** 多 Excel 文件列名不一致 **解决：** 建立完整的列名映射表，实现列名归一化函数
7. **问题：** 部分 Excel 文件缺少列 **解决：** 在合并前检查并添加缺失列，填充默认值'未知'
8. **问题：** 数据合并后存在重复记录 **解决：** 基于客户 ID+ 订单日期进行去重处理

6 实验心得

通过本次实验，我系统学习了金融数据分析的完整流程，取得了以下收获：

- 掌握了 Python、JavaScript 等编程语言的基本用法
- 熟悉了 Flask 框架的 Web 开发方法
- 学会了 pandas 等数据处理工具的使用，特别是 Excel 文件的读写和整合

- 理解了时间序列分析的基本概念，包括数据预处理、EDA 和预测建模
- 掌握了 LSTM 深度学习模型的构建方法，学会了与传统统计模型（SARIMA）进行对比
- 提升了问题排查和解决能力，特别是处理多源数据整合时的列名不一致问题

同时也认识到一些不足：

- 对深度学习理论理解不够深入，需要进一步学习
- 前端交互设计能力有待提高
- 代码注释和文档编写需要加强
- 测试用例覆盖不够全面

未来改进方向：学习更高级的时间序列预测模型（如 Transformer），优化前端用户体验，增加自动化测试覆盖，探索更多金融数据分析场景。